

INTERNET OF THINGS / LAPTOP LAB 2

Internet / Network Layer

IPv4, IPv6, routing, neighbor discovery, and path MTU

Estimated time: 120 minutes **Environment:** one Ubuntu 24.04 LTS VM or native Ubuntu.

Hardware: laptop only; no microcontroller, radio, or cloud account.

Learning outcomes

Build an isolated sensor–router–cloud topology; configure IPv4 and IPv6 addresses and routes; explain forwarding, next hops, ARP and IPv6 Neighbor Discovery; diagnose route and MTU failures using packet evidence.

1 Concepts and topology

IP moves packets between networks. A **subnet prefix** identifies which addresses are on a local link. A **route** selects an outgoing interface and possibly a **next hop**. A **router** forwards packets between links; enabling forwarding is separate from assigning addresses.

A Linux **network namespace** has its own interfaces, addresses, routes, and protocol state. A **veth pair** is a connected pair of virtual Ethernet interfaces. We use three namespaces, not three VMs. The namespace named **iot-cloud** is a local collector, not a real cloud account.

Namespace	Interface	IPv4	IPv6
iot-sensor	s0	10.10.1.2/24	2001:db8:1::2/64
iot-router	r0	10.10.1.1/24	2001:db8:1::1/64
iot-router	r1	10.10.2.1/24	2001:db8:2::1/64
iot-cloud	c0	10.10.2.2/24	2001:db8:2::2/64

s0 connects to r0; r1 connects to c0. The router has two interfaces because it joins two different subnets. IPv6 addresses from 2001:db8::/32 are for documentation; they are used only inside this isolated lab. Static routes are explicit in both directions. No NAT, DHCP, router advertisements, DNS, or external Internet route is needed.

Working environment and files

Use a single Ubuntu 24.04 LTS virtual machine (VM) on Windows or macOS, or run directly on Ubuntu. Allocate approximately 2 virtual CPUs, 4 GB RAM, and 25 GB disk space for a desktop VM. Select an Ubuntu image matching your laptop's CPU architecture. NAT networking is sufficient for installing packages; the lab traffic stays within the laptop. Keep all terminals inside the same VM.

Extract the companion ZIP, then open the matching `code/02_network` folder in a terminal. Alternatively, create an empty working folder and copy each complete program from the appendix into the indicated filename. For example, `nano filename.py` opens an editor; save with `Ctrl+O`, `Enter`, then `Ctrl+X`. Commands beginning with `sudo` ask for your Ubuntu password. Internet access is needed for installation, not for the experiments. Save results before cleanup. This tutorial can be completed independently of the other three.

```
sudo apt update
sudo apt install -y iproute2 iputils-ping tcpdump python3
ip -V
```

This lab needs Linux network namespaces and administrative access inside your own VM. A restricted

container or a managed online notebook is not equivalent to the VM. WSL environments can differ; use a full Ubuntu VM if namespace creation is unavailable. Use the provided `network_lab.sh`, or copy it from the appendix. It changes only its three namespaces.

2 Experiment 1: build and inspect the topology (20 minutes)

```
sudo bash network_lab.sh up
sudo ip netns list
sudo ip -n iot-sensor -br addr
sudo ip -n iot-router -br addr
sudo ip -n iot-cloud -br addr
sudo ip -n iot-sensor route
sudo ip -n iot-sensor -6 route
```

The script brings loopback and veth interfaces up, assigns both IP versions, enables IPv4/IPv6 forwarding in the router, and installs forward and return routes. The `nodad` flag skips IPv6 Duplicate Address Detection only for these fixed, unique, isolated lab addresses; normal networks should retain DAD.

Test a local next hop, then the remote endpoint:

```
sudo ip netns exec iot-sensor ping -c 3 10.10.1.1
sudo ip netns exec iot-sensor ping -c 3 10.10.2.2
sudo ip netns exec iot-sensor ping -6 -c 3 2001:db8:2::2
sudo ip -n iot-sensor route get 10.10.2.2
sudo ip -n iot-sensor -6 route get 2001:db8:2::2
```

Checkpoint. Both remote pings succeed. The route lookup selects `s0` and the router's sensor-side address as the next hop.

Record and explain. Explain why the sensor does not ARP for 10.10.2.2. Identify which router interface receives the request and which forwards it.

3 Experiment 2: observe ARP and IPv6 Neighbor Discovery (20 minutes)

In terminal A, capture on the sensor's interface:

```
sudo ip netns exec iot-sensor tcpdump -i s0 -nn -e -vv \
  'arp or icmp6'
```

In terminal B, flush only this namespace's neighbor cache and repeat both pings:

```
sudo ip -n iot-sensor neigh flush dev s0
sudo ip -n iot-sensor -6 neigh flush dev s0
sudo ip netns exec iot-sensor ping -c 2 10.10.2.2
sudo ip netns exec iot-sensor ping -6 -c 2 2001:db8:2::2
sudo ip -n iot-sensor neigh show
sudo ip -n iot-sensor -6 neigh show
```

IPv4 ARP resolves a local next-hop IP address to a link-layer address. IPv6 uses ICMPv6 Neighbor Solicitation and Neighbor Advertisement rather than ARP. Link-local IPv6 addresses beginning with `fe80::` may also appear; they are scoped to one link and often need an interface identifier in commands.

Checkpoint. Identify an ARP request/reply and an IPv6 neighbor-discovery exchange. The resolved next hop is the router, even though the end-to-end IP destination is the cloud endpoint. Stop the capture with Ctrl+C.

4 Experiment 3: forwarding and a missing return route (20 minutes)

First demonstrate hop limits:

```
sudo ip netns exec iot-sensor ping -t 1 -c 2 10.10.2.2
sudo ip netns exec iot-sensor ping -6 -t 1 -c 2 2001:db8:2::2
```

The router should return Time Exceeded. A hop limit of 1 cannot cross it; the field prevents packets from circulating forever. IPv4 calls the field TTL.

Temporarily remove only the IPv4 return route on the cloud endpoint:

```
sudo ip -n iot-cloud route del 10.10.1.0/24
sudo ip netns exec iot-sensor ping -c 3 -W 1 10.10.2.2
sudo ip netns exec iot-sensor ping -6 -c 3 2001:db8:2::2
# Restore immediately after recording the results
sudo ip -n iot-cloud route add 10.10.1.0/24 via 10.10.2.1
```

IPv4 should fail while IPv6 still works. Depending on reverse-path filtering, IPv4 requests may be discarded at the receiver or arrive without a usable reply route. Capture on c0 and inspect routes to distinguish the cases.

As a second fault, disable and restore router forwarding:

```
sudo ip netns exec iot-router sysctl -w net.ipv4.ip_forward=0
sudo ip netns exec iot-sensor ping -c 2 -W 1 10.10.2.2
sudo ip netns exec iot-router sysctl -w net.ipv4.ip_forward=1
```

Record and explain. Explain why correct endpoint addresses alone are insufficient. Show evidence of recovery after each restoration. Do not change host-wide sysctls.

5 Experiment 4: a constrained path MTU (25 minutes)

The **MTU** is the maximum IP packet size carried by a link without IP fragmentation. **Path MTU** is the smallest link MTU along a route. Set the second link to 1280 bytes and keep the sensor link at 1500:

```
sudo ip -n iot-router link set r1 mtu 1280
sudo ip -n iot-cloud link set c0 mtu 1280
sudo ip -n iot-router link show r1
```

In A watch ICMPv6 control traffic on the first link. In B send a large probe, then one that fits:

```
# Terminal A
sudo ip netns exec iot-sensor tcpdump -i s0 -nn -vv icmp6
# Terminal B
sudo ip netns exec iot-sensor ping -6 -M do -s 1400 -c 2 2001:db8:2::2
sudo ip netns exec iot-sensor ping -6 -M do -s 1232 -c 2 2001:db8:2::2
```

For these probes, IPv6 adds 40 bytes and ICMPv6 adds 8 bytes. Therefore $1232 + 40 + 8 = 1280$ fits; $1400 + 40 + 8 = 1448$ does not. Expect a Packet Too Big response from the router on the first oversized attempt. Subsequent oversized attempts may be rejected locally using cached path-MTU information. Such a local error is also a valid observation, not proof that the test failed.

IPv6 routers do not fragment forwarded packets. IPv6 requires links to support an MTU of at least 1280 bytes; smaller link technologies need an adaptation mechanism below IPv6. For comparison, try IPv4 with the Don't Fragment behavior:

```
sudo ip netns exec iot-sensor ping -4 -M do -s 1400 -c 2 10.10.2.2
sudo ip netns exec iot-sensor ping -4 -M do -s 1252 -c 2 10.10.2.2
```

Without IP options, IPv4 adds 20 bytes and ICMP adds 8, giving $1252 + 20 + 8 = 1280$. The oversized probe should fail with fragmentation-needed information or a cached local MTU error. Stop the capture.

Connection to constrained IoT

6LoWPAN means IPv6 over Low-Power Wireless Personal Area Networks. It provides adaptation such as header compression and fragmentation below IPv6 for small link frames. IEEE 802.15.4 is not Wi-Fi; the classic 127-byte 802.15.4 frame is not a 127-byte IPv6 MTU. A **border router** connects an IoT network to another IP network. Our router teaches the forwarding role, but this Ethernet lab does not emulate 6LoWPAN compression, RPL routing, or an 802.15.4 radio.

6 Experiment 5: reach a service over IPv6 (15 minutes)

Restore the MTUs. Then start a local HTTP collector inside the cloud namespace:

```
sudo ip -n iot-router link set r1 mtu 1500
sudo ip -n iot-cloud link set c0 mtu 1500
# Terminal A
sudo ip netns exec iot-cloud python3 -m http.server 8000 \
  --bind 2001:db8:2::2
```

In B, make an IPv6 TCP connection without depending on DNS or a proxy:

```
sudo ip netns exec iot-sensor python3 - <<'PY'
import socket
with socket.socket(socket.AF_INET6, socket.SOCK_STREAM) as s:
    s.settimeout(3)
    s.connect(("2001:db8:2::2", 8000))
    s.sendall(b"GET / HTTP/1.0\r\nHost: iot-cloud\r\n\r\n")
    print(s.recv(4096).decode(errors="replace"))
PY
```

Checkpoint. The response begins with an HTTP success status. This verifies routing beneath TCP and an application service. A ping alone cannot prove that a particular application is listening. Stop the server with Ctrl+C.

7 Analysis and submission (20 minutes)

Complete a table with the following columns: test, expected next hop, observed result, evidence, and explanation. Include baseline IPv4/IPv6, missing IPv4 return route, disabled forwarding, hop limit 1, and all four MTU probes.

1. Which addresses change at each routed hop: source/destination IP or source/destination link-layer addresses? Explain for this non-NAT topology.
2. Why must the cloud endpoint know a route back to the sensor?
3. Why can blocking all ICMPv6 break more than ping?
4. A UDP payload uses an 8-byte UDP header. What maximum UDP payload fits in a 1280-byte IPv6 packet with no extension headers? State all assumptions.
5. Explain the difference between an Ethernet MTU, a radio frame size, and 6LoWPAN adaptation-layer fragmentation.

Submit the report, address/route outputs, annotated packet evidence, and your copy of the setup

script. Explain actual observations if they differ because of a cached PMTU or local filtering.

Troubleshooting and cleanup

Operation not permitted: run inside a full Ubuntu VM with `sudo`. **Namespace exists:** stop old lab processes and use `down` before `up`. **IPv6 fails:** inspect `ip -n iot-sensor -6 addr`, both IPv6 routes, and `net.ipv6.conf.all.forwarding` inside the router. **No neighbor exchange:** flush the relevant namespace cache and retry. **Small probes fail:** first restore routes and forwarding; MTU cannot explain every connectivity failure.

```
# Stop capture/server terminals first, then:
sudo bash network_lab.sh down
sudo ip netns list
```

The script refuses cleanup while processes still occupy a lab namespace. Remove only the three lab namespaces; leave other namespaces untouched.

References

1. iproute2 manual, *ip-netns*. <https://man7.org/linux/man-pages/man8/ip-netns.8.html>
2. IETF, RFC 8200, *Internet Protocol, Version 6 (IPv6) Specification*, especially Sections 3–5. <https://www.rfc-editor.org/rfc/rfc8200.html>
3. IETF, RFC 8201, *Path MTU Discovery for IP version 6*. <https://www.rfc-editor.org/info/rfc8201>
4. IETF, RFC 4861, *Neighbor Discovery for IP version 6*. <https://www.rfc-editor.org/info/rfc4861>
5. IETF, RFC 4944, *Transmission of IPv6 Packets over IEEE 802.15.4 Networks*. <https://www.rfc-editor.org/info/rfc4944>

A Complete source: network_lab.sh

```
#!/usr/bin/env bash
# Isolated, static, dual-stack network. Run inside an Ubuntu VM.
set -euo pipefail
names=(iot-sensor iot-router iot-cloud)
if [[ $EUID -ne 0 ]]; then
    echo "Run: sudo bash network_lab.sh up|down" >&2
    exit 1
fi
exists() { ip netns list | awk '{print $1}' | grep -qx "$1"; }
remove_lab() {
    for ns in "${names[@]"; do
        if exists "$ns"; then ip netns del "$ns"; fi
    done
}
case "${1:-}" in
    down)
        for ns in "${names[@]"; do
            if exists "$ns" && [[ -n $(ip netns pids "$ns") ]]; then
                echo "Stop processes in $ns before cleanup." >&2
                exit 1
            fi
        done
        remove_lab
        exit 0
    ;;
    up) ;;
    *) echo "Usage: sudo bash network_lab.sh up|down"; exit 1 ;;
esac
for ns in "${names[@]"; do
    if exists "$ns"; then
```

```
    echo "$ns already exists; clean up the previous lab first." >&2
    exit 1
fi
done
trap remove_lab ERR
for ns in "${names[@]"; do
    ip netns add "$ns"
    ip -n "$ns" link set lo up
done
ip -n iot-sensor link add s0 type veth peer name r0 netns iot-router
ip -n iot-router link add r1 type veth peer name c0 netns iot-cloud
ip -n iot-sensor addr add 10.10.1.2/24 dev s0
ip -n iot-router addr add 10.10.1.1/24 dev r0
ip -n iot-router addr add 10.10.2.1/24 dev r1
ip -n iot-cloud addr add 10.10.2.2/24 dev c0
ip -n iot-sensor -6 addr add 2001:db8:1::2/64 dev s0 nodad
ip -n iot-router -6 addr add 2001:db8:1::1/64 dev r0 nodad
ip -n iot-router -6 addr add 2001:db8:2::1/64 dev r1 nodad
ip -n iot-cloud -6 addr add 2001:db8:2::2/64 dev c0 nodad
ip -n iot-sensor link set s0 up
ip -n iot-router link set r0 up
ip -n iot-router link set r1 up
ip -n iot-cloud link set c0 up
ip netns exec iot-router sysctl -qw net.ipv4.ip_forward=1
ip netns exec iot-router sysctl -qw net.ipv6.conf.all.forwarding=1
ip -n iot-sensor route add 10.10.2.0/24 via 10.10.1.1
ip -n iot-cloud route add 10.10.1.0/24 via 10.10.2.1
ip -n iot-sensor -6 route add 2001:db8:2::/64 via 2001:db8:1::1
ip -n iot-cloud -6 route add 2001:db8:1::/64 via 2001:db8:2::1
trap - ERR
echo "READY: iot-sensor <-> iot-router <-> iot-cloud"
```